

REPORT DI PROGETTO: PSD 2025-2026

TRACCIA 1: Sistema di gestione degli interventi di manutenzione in un condominio
Componenti del Gruppo: Giosuè Senatore, Jacopo Salimbene, Manuel Vitolo

1. Ambiente di Sviluppo e Compilazione

Il progetto è stato sviluppato seguendo lo standard **C99** per garantire la compatibilità con i principali compilatori (GCC/Clang). Il sistema è provvisto di un **Makefile** che automatizza il processo di linking e compilazione. Il progetto è stato testato e risulta compilabile e funzionante su ambiente Windows tramite WSL (Ubuntu) e MacOS. (Li abbiamo provati su tutti e due i sistemi operativi).

2. Come usare e interagire con il sistema

L'i

nterazione con il sistema avviene tramite un'Interfaccia a Riga di Comando (CLI) testuale guidata da un menu principale. All'avvio dell'eseguibile, le strutture dati vengono allocate dinamicamente. L'utente (l'amministratore del condominio) può selezionare le operazioni digitando il numero corrispondente (da 1 a 0):

- **Operazioni di Inserimento (1-2):** Permettono di registrare nuove richieste (assegnando un'urgenza) e inserire nuovi tecnici specificando le loro competenze.
- **Operazioni di Visualizzazione (3-4, 8-9):** Consentono di stampare a schermo le code di priorità attive, l'albero di storico degli interventi passati, il carico di lavoro per ogni tecnico e di generare un report numerico riassuntivo.
- **Operazioni di Gestione (5-7):** Il cuore operativo. Permettono di assegnare un tecnico a una richiesta (verificando disponibilità e conflitti), far avanzare lo stato (es. da "In Lavorazione" a "Conclusa") e ricercare pratiche specifiche tramite ID.
- **Uscita (0):** Termina il programma invocando il modulo di *Garbage Collection manuale*, che naviga tutte le strutture deallocando la memoria per evitare *memory leaks*.

3. Motivazione scelta dell' ADT (Abstract Data Type)

La selezione delle strutture dati è stata guidata dalla necessità di ottimizzare le complessità temporali per i casi d'uso specifici di un condominio.

1. Gestione delle Richieste Attive (Max-Heap / Coda di Priorità):

- *Motivazione:* Le emergenze assolute (es. tubo rotto) devono avere la precedenza su guasti minori (es. lampadina fulminata), indipendentemente dall'ordine di arrivo.
- *Scelta:* Coda di Priorità basata su un array strutturato come Max-Heap. Garantisce che la richiesta più urgente sia sempre alla radice (indice 0) ed estraibile in tempo $O(1)$. L'inserimento si auto-bilancia in tempo logaritmico $O(\log N)$, molto più efficiente di una lista ordinata.

2. Gestione dei Tecnici (Tabella Hash):

- *Motivazione:* Rintracciare istantaneamente il profilo di un tecnico partendo dall'ID senza scorrere un'anagrafica lineare.
 - *Scelta:* Tabella Hash con gestione delle collisioni tramite liste concatenate (*chaining*). Ricerca, accesso e assegnazione avvengono in tempo costante $O(1)$ nel caso medio.
3. **Pianificazione e Agende (Lista Concatenata Semplice):**
- *Motivazione:* Verificare conflitti di orario per un tecnico nella stessa giornata.
 - *Scelta:* Poiché l'agenda attiva di un singolo tecnico contiene pochi appuntamenti, una Lista Concatenata Semplice permette inserimenti in testa in $O(1)$ e uno scorrimento per controlli di overlap temporale rapido e leggero in memoria.
4. **Storico degli Interventi (Albero Binario di Ricerca - BST):**
- *Motivazione:* L'archivio delle pratiche concluse è destinato a crescere a dismisura. Serve una ricerca veloce e una stampa ordinata.
 - *Scelta:* Albero Binario di Ricerca (BST). Dimezza lo spazio di ricerca a ogni passo garantendo query in $O(\log N)$. Consente inoltre la generazione di report ordinati tramite visita In-Order.

4. Progettazione: Architettura del Sistema

Il sistema segue un'architettura modulare *Multi-file* divisa nei seguenti moduli funzionali per garantire incapsulamento e manutenibilità:

- **Modulo Richieste (*richieste.h/.c*):** Gestisce l'inserimento nel Max-Heap, l'estrazione delle emergenze e la funzione di riordino *heapifyUp*.
- **Modulo Tecnici (*tecnici.h/.c*):** Gestisce l'anagrafica e l'indicizzazione in $O(1)$ tramite la Tabella Hash e il calcolo dell'indice.
- **Modulo Storico (*storico.h/.c*):** Dedicato all'archiviazione a lungo termine. Contiene le logiche di inserimento, ricerca ricorsiva e deallocazione Post-Order del BST.
- **Modulo Pianificazione (*pianificazione.h/.c*):** Il "motore" transazionale. Preleva dati da Heap e Hash, esegue le validazioni di business (competenze/conflitti) e aggiorna i grafi logici.
- **Modulo Main (*main.c*):** Implementa il loop applicativo, il menu CLI, e funge da *controller* principale che orchestra l'inizializzazione e la distruzione delle variabili di ambiente.

5. Specifica Sintattica e Semantica

Di seguito le specifiche delle operazioni fondamentali (Core API) del sistema, inclusi effetti collaterali e pre/post condizioni.

5.1 Inserimento Richiesta (Max-Heap)

- **Sintassi:** `void inserisciRichiestaHeap(SistemaCondominio* sys, Richiesta* nuova)`
- **Pre-condizioni:** `sys` inizializzato. Dimensione attuale dell'Heap strettamente minore di `MAX_HEAP_CAPACITY`.
- **Post-condizioni:** La richiesta è inserita e l'albero è ribilanciato in base alla priorità `urgenza_val`.

- **Effetti Collaterali:** Variabile `heap_size` incrementata di 1.

5.2 Registrazione Tecnico (Hash Table)

- **Sintassi:** `void registraTecnicoHash(SistemaCondominio* sys, int id, const char* nome, const char* spec, bool disp)`
- **Pre-condizioni:** L'ID fornito deve essere intero e positivo. La tabella Hash in `sys` deve essere pre-inizializzata a `NULL`.
- **Post-condizioni:** Struttura allocata in memoria e posizionata nell'indice calcolato da `id % HASH_SIZE`. In caso di collisione, nodo inserito in testa alla lista. Accesso $O(1)$.
- **Effetti Collaterali:** Mutazione dei puntatori nel bucket corrispondente di `hash_tecnici`.

5.3 Pianificazione Intervento

- **Sintassi:** `bool pianificaIntervento(SistemaCondominio* sys, int id_richiesta, int id_tecnico, Data d, FasciaOraria fascia)`
- **Pre-condizioni:** Entrambi gli ID devono essere validi. Tecnico `disponibile == true`. Specializzazione del tecnico coincidente con tipologia richiesta.
- **Post-condizioni:** Restituisce `true` se l'agenda non presenta sovrapposizioni temporali e lega i dati. Restituisce `false` se fallisce le validazioni.
- **Effetti Collaterali:** Stato della richiesta settato a `PIANIFICATA`. Nuovo nodo creato e inserito nell'agenda (linked list) del tecnico.

5.4 Archiviazione Storica

- **Sintassi:** `void archiviaNelloStorico(SistemaCondominio* sys, Richiesta* r)`
- **Pre-condizioni:** Richiesta giunta allo stato `CONCLUSA` o `ANNULLATA`.
- **Post-condizioni:** La richiesta è rimossa logicamente dall'array Heap (tramite rimpiazzo e riduzione size) e il suo puntatore è inserito come foglia nel BST preservando l'ordinamento numerico dell'ID.
- **Effetti Collaterali:** Diminuzione `heap_size`. Altezza dell'albero `bst_storico_root` potenzialmente incrementata.

6. Razionale e Casi di Test

Caso di Test	Obiettivo / Razionale	Esito Atteso / Risultato
1. Registrazione Richieste (Max-Heap)	Verificare che, inserendo richieste in ordine casuale, l'algoritmo <i>heapify</i> porti sempre alla radice (indice 0) la	La coda di priorità si auto-bilancia: l'emergenza viene processata per prima

	richiesta con urgenza CRITICA $O(\log N)$.	a prescindere dall'ID di arrivo.
2. Registrazione Tecnici (Hash)	Verificare la robustezza della funzione Hash inserendo intenzionalmente due tecnici (es. ID 105 e 205) che collidono nello stesso bucket (Indice 5).	La lista concatenata interna salva entrambi i record senza sovrascritture o segmentation fault (<i>Chaining</i> funzionante).
3. Assegnazione Corretta	Validare le regole di business garantendo che un "Elettricista" non possa essere assegnato a un guasto "Idraulico", e che tecnici in ferie vengano scartati.	Il sistema blocca la pianificazione restituendo un messaggio d'errore semantico e false .
4. Pianificazione e Conflitti	L'agenda (Lista) non deve permettere ubiquità. Se un tecnico è occupato dalle 9:00 alle 11:00, un incarico per le 10:00 deve essere respinto.	La logica di intersezione segmenti rileva la sovrapposizione matematica; assegnazione bloccata con errore.
5. Aggiornamento Stato	Testare lo spostamento transazionale dei puntatori senza perdite di memoria: quando lo stato diventa CONCLUSA , l'elemento migra dall'array alla struttura ad albero.	La richiesta scompare fisicamente dal pool "Active" ed è storicizzata permanentemente nel BST.
6. Ricerca e Filtri	Confrontare l'efficienza: cercare una pratica recente scorre l'Heap $O(N)$, mentre cercare una pratica passata sfrutta il partizionamento del BST $O(\log N)$.	Il sistema individua dinamicamente in quale Database si trova l'ID e stampa i dettagli corretti.

7. Storico Interventi (BST)	Confermare che l'albero garantisca nativamente l'ordinamento. Inserendo ID disordinati (es. 5, 2, 8), la stampa deve rispecchiare una sequenza crescente.	La funzione ricorsiva stampaBST esegue una visita <i>In-Order</i> restituendo l'archivio perfettamente numerato (2, 5, 8).
8. Generazione Report	Assicurare che le funzioni di calcolo metrico (es. conteggio nodi albero) funzionino contando ricorsivamente ogni ramo senza saltare dati.	Il report stampa il valore esatto di pratiche aperte lette dall'Heap e archiviate lette dal BST.